Page URL: (private repository, not public)

shorts-auto – complete upload script

API client: shorts-auto (private, single user). **Use case:** Video Uploading & Account Management.

This is the complete, working code that performs every upload. There is no user interface: the script runs once a day on a schedule (GitHub Actions) and uploads the owner's own finished videos to the owner's own four YouTube channels with `videos.insert`. It was used to upload these two test videos, which are currently locked as private because the project has not been audited yet: youtube.com/shorts/GklHx3Ic-Tw and youtube.com/shorts/Iwne3VF2yrA.

Flow: 1) the owner authorizes each of his channels once through the Google OAuth 2.0 consent screen (scopes youtube.upload, youtube.readonly, yt-analytics.readonly); 2) the refresh tokens are stored as encrypted repository secrets; 3) every day `publicar_dia.py` picks the next approved video of each series and calls `subir.py`, which sends it with `videos.insert` (title, description, tags, language, privacy status, made-for-kids = false, containsSyntheticMedia = true); 4) the script prints the URL of the uploaded video. Comments in the code are in Spanish.

motor/subir.py

```
"""Sube un Short a YouTube con la API oficial.

Uso: python3 subir.py salida/ep001_es.mp4 salida/ep001_es.json CANAL
CANAL es el sufijo del secreto con el refresh token, p. ej. TONI_ES -> YT_TOKEN_TONI_ES
Variables: YT_CLIENT_ID, YT_CLIENT_SECRET, YT_TOKEN_<CANAL>, YT_PRIVACIDAD (private | public)
"""
import json
import os
import sys

from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build
from googleapiclient.http import MediaFileUpload


def subir(mp4, meta_json, canal, publicar_a=None):
    """publicar_a: hora ISO en UTC (2026-11-13T14:00:00Z) para dejarlo programado; YouTube lo
exige en privado hasta entonces."""
    meta= json.load(open(meta_json, encoding='utf-8'))
    cred= Credentials(None, refresh_token=os.environ[f'YT_TOKEN_{canal}'],
client_id=os.environ['YT_CLIENT_ID'],
                      client_secret=os.environ['YT_CLIENT_SECRET'],
token_uri='https://oauth2.googleapis.com/token')
    yt= build('youtube', 'v3', credentials=cred)
    cuerpo= {
```

```
        'snippet': {'title': meta['titulo'][:100], 'description': meta['descripcion'], 'tags':
meta.get('tags', []),
                    'categoryId': '23' if canal.startswith('TONI') else '27',
                    'defaultLanguage': meta['idioma'], 'defaultAudioLanguage': meta['idioma']},
        'status': {'privacyStatus': os.environ.get('YT_PRIVACIDAD', 'private'),
'selfDeclaredMadeForKids': False,
                   'containsSyntheticMedia': True},  # marco siempre que es contenido generado
    }
    if publicar_a:
        cuerpo['status']['privacyStatus']= 'private'
        cuerpo['status']['publishAt']= publicar_a
    req= yt.videos().insert(part='snippet,status', body=cuerpo, media_body=MediaFileUpload(mp4,
chunksize=-1, resumable=True))
    resp= None
    while resp is None:
        _, resp= req.next_chunk()
    print('subido:', 'https://youtube.com/shorts/' + resp['id'])
    return resp['id']


if __name__ == '__main__':
    subir(sys.argv[1], sys.argv[2], sys.argv[3])
```

motor/publicar_dia.py

```
"""Publica el siguiente video aprobado de cada serie (uno al dia por canal).

cola.json lleva, en orden, los episodios con visto bueno; los mp4 estan en la rama listos.
estado.json lleva los que ya se subieron. No genera nada: solo sube lo ya aprobado.
"""
import datetime
import json
import os
import sys

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

RAIZ= os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SERIES= {'ep': 'TONI', 'ysi': 'YSI'}


def main():
    cola= json.load(open(os.path.join(RAIZ, 'cola.json')))
    ruta_estado= os.path.join(RAIZ, 'estado.json')
    estado= json.load(open(ruta_estado))
    for pref, canal in SERIES.items():
        hechos= estado.setdefault(pref, [])
        pendientes= [e for e in cola if e.startswith(pref) and e + '.json' not in hechos]
        if not pendientes:
            print(f'{pref}: no hay episodios aprobados en la cola')
            continue
        ep= pendientes[0]
        import subir
        for idi in ('es', 'en'):
            mp4= os.path.join(RAIZ, 'listos', f'{ep}_{idi}.mp4')
            # espanol sale al subir (13:00 de Madrid); ingles queda programado para las 15:00 de
Madrid (9:00 en la costa este)
            hora= None
            if idi == 'en' and os.environ.get('YT_PRIVACIDAD') == 'public':
                ahora= datetime.datetime.now(datetime.timezone.utc)
                cuando= ahora.replace(hour=14, minute=0, second=0, microsecond=0)
                hora= cuando.strftime('%Y-%m-%dT%H:%M:%SZ') if cuando > ahora else None
            subir.subir(mp4, mp4.replace('.mp4', '.json'), f'{canal}_{idi.upper()}', hora)
        hechos.append(ep + '.json')
```

```
        # guardo tras cada serie para no repetir una subida si falla la siguiente
        json.dump(estado, open(ruta_estado, 'w'), indent=1)
        print(f'{pref}: subido {ep}, quedan {len(pendientes) - 1} en la cola')


if __name__ == '__main__':
    main()
```

motor/subir_listo.py

```
"""Sube un video ya aprobado (rama listos) al canal que le toca, o solo comprueba las claves.

Uso: python3 subir_listo.py comprobar
     python3 subir_listo.py subir listos/ep001_es.mp4
El canal sale del nombre del archivo: ep -> TONI, ysi -> YSI, y el idioma del sufijo.
"""
import os
import sys

from google.oauth2.credentials import Credentials
from googleapiclient.discovery import build

CANALES= ['TONI_ES', 'TONI_EN', 'YSI_ES', 'YSI_EN']


def comprobar():
    # miro a que canal apunta cada clave, sin subir nada
    bien= True
    for canal in CANALES:
        try:
            cred= Credentials(None, refresh_token=os.environ[f'YT_TOKEN_{canal}'],
client_id=os.environ['YT_CLIENT_ID'],
                              client_secret=os.environ['YT_CLIENT_SECRET'],
token_uri='https://oauth2.googleapis.com/token')
            yt= build('youtube', 'v3', credentials=cred)
            r= yt.channels().list(part='snippet', mine=True).execute()
            print(f'{canal}: {r["items"][0]["snippet"]["title"]}')
        except Exception as e:
            bien= False
            print(f'{canal}: FALLO {type(e).__name__}: {str(e)[:200]}')
    return bien


def canal_de(mp4):
    nombre= os.path.basename(mp4).replace('.mp4', '')
    serie= 'TONI' if nombre.startswith('ep') else 'YSI'
    return f'{serie}_{nombre.split("_")[-1].upper()}'


if __name__ == '__main__':
    if sys.argv[1] == 'comprobar':
        sys.exit(0 if comprobar() else 1)
    import subir
    for mp4 in sys.argv[2:]:
        subir.subir(mp4, mp4.replace('.mp4', '.json'), canal_de(mp4))
```

.github/workflows/publicar_dia.yml

```yaml
name: Publicar el video del dia

on:
  schedule:
    - cron: '0 12 * * *'   # 13:00 hora de Madrid (invierno): 1 video al dia por canal
  workflow_dispatch:

permissions:
  contents: write

concurrency: publicar-dia

jobs:
  publicar:
    runs-on: ubuntu-latest
    env:
      YT_PRIVACIDAD: ${{ vars.YT_PRIVACIDAD || 'private' }}
      YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
      YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
      YT_TOKEN_TONI_ES: ${{ secrets.YT_TOKEN_TONI_ES }}
      YT_TOKEN_TONI_EN: ${{ secrets.YT_TOKEN_TONI_EN }}
      YT_TOKEN_YSI_ES: ${{ secrets.YT_TOKEN_YSI_ES }}
      YT_TOKEN_YSI_EN: ${{ secrets.YT_TOKEN_YSI_EN }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/checkout@v4
        with:
          ref: listos
          path: listos
      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
      - run: pip install -q google-api-python-client google-auth
      - name: Subir
        run: python3 motor/publicar_dia.py
      - name: Guardar estado
        if: always()
        run: |
          git config user.name "shorts-bot"
          git config user.email "shorts-bot@users.noreply.github.com"
          git add estado.json && git commit -m "Actualizar estado de publicacion" || true
          git push
```